

Assembleur

Generated by Doxygen 1.8.11

Contents

1	Data Structure Index	1
1.1	Data Structures	1
2	File Index	3
2.1	File List	3
3	Data Structure Documentation	5
3.1	t_label Struct Reference	5
3.1.1	Detailed Description	5
3.1.2	Field Documentation	5
3.1.2.1	addr	5
3.1.2.2	label	5
4	File Documentation	7
4.1	asm.c File Reference	7
4.1.1	Detailed Description	8
4.1.2	Function Documentation	9
4.1.2.1	AddInstruction(char *line)	9
4.1.2.2	addLabel(char *line, int addr)	9
4.1.2.3	Clean_Memory()	9
4.1.2.4	dumpBinaryCode()	9
4.1.2.5	dumpBinaryCode(char *filename)	9
4.1.2.6	generateBinary(FILE *fichier_dest)	9
4.1.2.7	main(int argc, char *argv[])	9
4.1.2.8	parseAsm(FILE *fichier_src)	9

4.1.2.9	resolveReferences()	9
4.1.3	Variable Documentation	9
4.1.3.1	buffer	9
4.1.3.2	codeSegment	10
4.1.3.3	currentInst	10
4.1.3.4	currentLabel	10
4.1.3.5	currentRef	10
4.1.3.6	tabLabels	10
4.1.3.7	tabReferences	10
4.2	asm.h File Reference	10
4.2.1	Detailed Description	11
4.2.2	Macro Definition Documentation	11
4.2.2.1	TAILLE_BUFFER	11
4.2.3	Function Documentation	11
4.2.3.1	AddInstruction(char *line)	11
4.2.3.2	addLabel(char *line, int addr)	12
4.2.3.3	Clean_Memory()	12
4.2.3.4	dumpBinaryCode(char *)	12
4.2.3.5	generateBinary(FILE *)	12
4.2.3.6	parseAsm(FILE *)	12
4.2.3.7	resolveReferences()	12
4.3	debug.h File Reference	12
4.3.1	Detailed Description	13
4.3.2	Macro Definition Documentation	13
4.3.2.1	DEBUG	13
4.4	Instructions.h File Reference	13
4.4.1	Detailed Description	14
4.4.2	Macro Definition Documentation	14
4.4.2.1	I_ADD	14
4.4.2.2	I_AND	14

4.4.2.3	I_DIV	15
4.4.2.4	I_EQ	15
4.4.2.5	I_EQI	15
4.4.2.6	I_EQS	15
4.4.2.7	I_HALT	15
4.4.2.8	I_J	15
4.4.2.9	I_JF	15
4.4.2.10	I_MULT	15
4.4.2.11	I_NOT	16
4.4.2.12	I_OR	16
4.4.2.13	I_POP	16
4.4.2.14	I_PUSH	16
4.4.2.15	I_PUSHI	16
4.4.2.16	I_SUB	16
4.4.2.17	I_XOR	16
Index		17

Chapter 1

Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

t_label	Structure utilisée pour la résolution des références	5
-------------------------	--	-------------------

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

asm.c	Regroupe les fonctions permettant de convertir un fichier assembleur .asm en un fichier .bin (utilisé dans la machine virtuelle)	7
asm.h	Regroupe les fonctions permettant de convertir un fichier assembleur en un fichier .o	10
debug.h	Librairie dédiée à la déclaration d'une variable DEBUG pour activer/désactiver le mode DEBUG	12
Instructions.h	Regroupe toutes les instructions utiles pour l'exécution du compilateur	13

Chapter 3

Data Structure Documentation

3.1 t_label Struct Reference

Structure utilisée pour la résolution des références.

Data Fields

- char * [label](#)
- int [addr](#)

3.1.1 Detailed Description

Structure utilisée pour la résolution des références.

Definition at line 72 of file asm.c.

3.1.2 Field Documentation

3.1.2.1 int addr

Definition at line 74 of file asm.c.

3.1.2.2 char* label

Definition at line 73 of file asm.c.

The documentation for this struct was generated from the following file:

- [asm.c](#)

Chapter 4

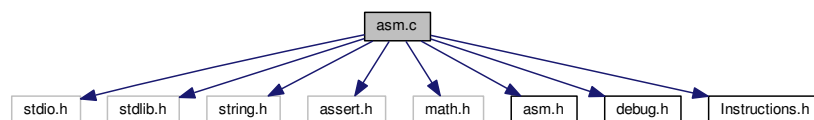
File Documentation

4.1 asm.c File Reference

Regroupe les fonctions permettant de convertir un fichier assembleur .asm en un fichier .bin (utilisé dans la machine virtuelle)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <assert.h>
#include <math.h>
#include "asm.h"
#include "debug.h"
#include "Instructions.h"
```

Include dependency graph for asm.c:



Data Structures

- struct [t_label](#)

Structure utilisée pour la résolution des références.

Functions

- void [AddInstruction](#) (char *line)
- void [addLabel](#) (char *line, int addr)
- void [dumpBinaryCode](#) ()
- void [parseAsm](#) (FILE *fichier_src)
- void [resolveReferences](#) ()
- void [generateBinary](#) (FILE *fichier_dest)
- void [dumpBinaryCode](#) (char *filename)
- void [Clean_Memory](#) ()
- int [main](#) (int argc, char *argv[])

Variables

- int `currentLabel` = 0
- int `currentInst` = 0
- int `currentRef` = 0
- int * `codeSegment` = NULL
- char `buffer` [TAILLE_BUFFER]
- t_label * `tabLabels` = NULL
- t_label * `tabReferences` = NULL

4.1.1 Detailed Description

Regroupe les fonctions permettant de convertir un fichier assembleur .asm en un fichier .bin (utilisé dans la machine virtuelle)

assembleur.c

Author

Jeanne Baumier & Thomas Broussard

Date

6 février 2017

Version

2

Date

12/03/17

Ajout de la conversion bin vers asm pour vérifier le bon déroulement de la conversion (voir fonction `dumpBinary↔Code`)

Les tableaux de structures sont à présent dynamique, ce qui rend le programme beaucoup plus flexible

Ajout d'une fonction effaçant les espaces mémoires alloués dynamiquement

La lecture du fichier source ligne par ligne (dans `parseASM`) passe à présent par une chaîne de caractère Instruction de taille variable

-> Intérêt : les fonctions prenant en paramètre la chaîne Instruction ne manipuleront plus une chaîne de taille fixe , pouvant contenir un nombre important

Version

1

Date

11/03/17

Ajout de la 2ème parse permettant de compléter les références indéfinies

Régulation de l'espace mémoire alloué dynamiquement : les fuites mémoires ont été corrigées via la commande `valgrind`

Version

0

Date

07/03/17

Refonte complète du fichier Assembleur.c pour répondre aux attentes du projet

4.1.2 Function Documentation

4.1.2.1 void AddInstruction (char * *line*)

Definition at line 230 of file asm.c.

4.1.2.2 void addLabel (char * *line*, int *addr*)

Definition at line 465 of file asm.c.

4.1.2.3 void Clean_Memory ()

Definition at line 668 of file asm.c.

4.1.2.4 void dumpBinaryCode ()

4.1.2.5 void dumpBinaryCode (char * *filename*)

Definition at line 507 of file asm.c.

4.1.2.6 void generateBinary (FILE * *fichier_dest*)

Definition at line 204 of file asm.c.

4.1.2.7 int main (int *argc*, char * *argv*[])

Definition at line 689 of file asm.c.

4.1.2.8 void parseAsm (FILE * *fichier_src*)

Definition at line 113 of file asm.c.

4.1.2.9 void resolveReferences ()

Definition at line 175 of file asm.c.

4.1.3 Variable Documentation

4.1.3.1 char buffer[TAILLE_BUFFER]

Buffer utilisable pour créer des chaines de caractères dynamique, avec l'utilisation de fgets ou sscanf

Definition at line 62 of file asm.c.

4.1.3.2 `int* codeSegment = NULL`

Tableau utile pour la génération des fichiers de destination

Definition at line 59 of file asm.c.

4.1.3.3 `int currentInst = 0`

Variable liée à void [addLabel\(char *line, int addr\)](#);

Definition at line 54 of file asm.c.

4.1.3.4 `int currentLabel = 0`

Variable liée à void [addLabel\(char *line, int addr\)](#);

Definition at line 52 of file asm.c.

4.1.3.5 `int currentRef = 0`

Variable liée à void [decodeInstruction\(char *line\)](#);

Definition at line 56 of file asm.c.

4.1.3.6 `t_label* tabLabels = NULL`

Structure globale *tabLabels

Definition at line 78 of file asm.c.

4.1.3.7 `t_label* tabReferences = NULL`

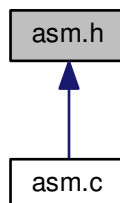
Structure globale *tabReferences

Definition at line 80 of file asm.c.

4.2 asm.h File Reference

Regroupe les fonctions permettant de convertir un fichier assembleur en un fichier .o.

This graph shows which files directly or indirectly include this file:



Macros

- #define [TAILLE_BUFFER](#) 1000

Functions

- void [parseAsm](#) (FILE *)
- void [resolveReferences](#) ()
- void [generateBinary](#) (FILE *)
- void [AddInstruction](#) (char *line)
- void [addLabel](#) (char *line, int addr)
- void [dumpBinaryCode](#) (char *)
- void [Clean_Memory](#) ()

4.2.1 Detailed Description

Regroupe les fonctions permettant de convertir un fichier assembleur en un fichier .o.

[asm.h](#)

Author

Jeanne Baumier & Thomas Broussard

Date

6 février 2017

Version

1

Date

11/02/17

Suppression des instructions présentes dans le programme (remplacé par instructions.h)

4.2.2 Macro Definition Documentation

4.2.2.1 #define [TAILLE_BUFFER](#) 1000

Definition at line 24 of file asm.h.

4.2.3 Function Documentation

4.2.3.1 void [AddInstruction](#) (char * *line*)

Definition at line 230 of file asm.c.

4.2.3.2 void addLabel (char * *line*, int *addr*)

Definition at line 465 of file asm.c.

4.2.3.3 void Clean_Memory ()

Definition at line 668 of file asm.c.

4.2.3.4 void dumpBinaryCode (char *)

Definition at line 507 of file asm.c.

4.2.3.5 void generateBinary (FILE *)

Definition at line 204 of file asm.c.

4.2.3.6 void parseAsm (FILE *)

Definition at line 113 of file asm.c.

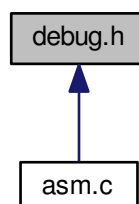
4.2.3.7 void resolveReferences ()

Definition at line 175 of file asm.c.

4.3 debug.h File Reference

Librairie dédiée à la déclaration d'une variable DEBUG pour activer/désactiver le mode DEBUG.

This graph shows which files directly or indirectly include this file:



Macros

- `#define DEBUG 0`

4.3.1 Detailed Description

Librairie dédiée à la déclaration d'une variable DEBUG pour activer/désactiver le mode DEBUG.

[debug.h](#)

Author

Jeanne Baumier & Thomas Broussard

Date

6 février 2017

4.3.2 Macro Definition Documentation

4.3.2.1 `#define DEBUG 0`

Variable de DEBUG :

0 : DEBUG désactivé

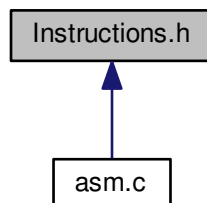
1 : DEBUG activé

Definition at line 19 of file debug.h.

4.4 Instructions.h File Reference

Regroupe toutes les instructions utiles pour l'exécution du compilateur.

This graph shows which files directly or indirectly include this file:



Macros

- `#define I_ADD 0`
- `#define I_MULT 1`
- `#define I_SUB 2`
- `#define I_DIV 3`
- `#define I_EQ 12`
- `#define I_EQS 13`
- `#define I_EQI 14`
- `#define I_AND 20`
- `#define I_NOT 21`
- `#define I_OR 22`
- `#define I_XOR 23`
- `#define I_PUSHI 100`
- `#define I_PUSH 101`
- `#define I_POP 102`
- `#define I_JF 200`
- `#define I_J 201`
- `#define I_HALT 400`

4.4.1 Detailed Description

Regroupe toutes les instructions utiles pour l'exécution du compilateur.

[Instructions.h](#)

Author

Jeanne Baumier & Thomas Broussard

Date

6 février 2017

4.4.2 Macro Definition Documentation

4.4.2.1 `#define I_ADD 0`

Effectue une addition entre deux valeurs situées au sommet de la pile

Definition at line 16 of file Instructions.h.

4.4.2.2 `#define I_AND 20`

Effectue un ET entre deux valeurs situées au sommet de la pile ('1' ou '0')

Definition at line 30 of file Instructions.h.

4.4.2.3 #define I_DIV 3

Effectue une division entre deux valeurs situées au sommet de la pile et garde l'entier supérieur

Definition at line 22 of file Instructions.h.

4.4.2.4 #define I_EQ 12

Vérifie si les deux valeurs au sommet de la pile sont égales ou non (renvoie '1' ou '0')

Definition at line 24 of file Instructions.h.

4.4.2.5 #define I_EQI 14

Vérifie si la valeur au sommet de la pile est inférieure à la valeur en dessous dans la pile (renvoie '1' ou '0')

Definition at line 28 of file Instructions.h.

4.4.2.6 #define I_EQS 13

Vérifie si la valeur au sommet de la pile est supérieure à la valeur en dessous dans la pile (renvoie '1' ou '0')

Definition at line 26 of file Instructions.h.

4.4.2.7 #define I_HALT 400

Indique la fin du programme

Definition at line 48 of file Instructions.h.

4.4.2.8 #define I_J 201

Permet d'exécuter un Jump inconditionnel

Definition at line 46 of file Instructions.h.

4.4.2.9 #define I_JF 200

Permet d'exécuter un Jump si la condition qui le précède est fausse

Definition at line 44 of file Instructions.h.

4.4.2.10 #define I_MULT 1

Effectue une multiplication entre deux valeurs situées au sommet de la pile

Definition at line 18 of file Instructions.h.

4.4.2.11 #define I_NOT 21

Inverse la valeur au sommet de la pile ('1' ou '0')

Definition at line 32 of file Instructions.h.

4.4.2.12 #define I_OR 22

Effectue un OU entre deux valeurs situées au sommet de la pile ('1' ou '0')

Definition at line 34 of file Instructions.h.

4.4.2.13 #define I_POP 102

Sauvegarde la valeur au sommet de la pile dans le tableau de variable

Definition at line 42 of file Instructions.h.

4.4.2.14 #define I_PUSH 101

Ajoute une valeur contenue dans le tableau de variable dans la pile d'exécution

Definition at line 40 of file Instructions.h.

4.4.2.15 #define I_PUSHI 100

Ajoute une valeur immédiate dans la pile d'exécution

Definition at line 38 of file Instructions.h.

4.4.2.16 #define I_SUB 2

Effectue une soustraction entre deux valeurs situées au sommet de la pile

Definition at line 20 of file Instructions.h.

4.4.2.17 #define I_XOR 23

Effectue un OU EXCLUSIF entre deux valeurs situées au sommet de la pile ('1' ou '0')

Definition at line 36 of file Instructions.h.

Index

AddInstruction
asm.c, 9
asm.h, 11

addLabel
asm.c, 9
asm.h, 11

addr
t_label, 5

asm.c, 7
AddInstruction, 9
addLabel, 9
buffer, 9
Clean_Memory, 9
codeSegment, 9
currentInst, 10
currentLabel, 10
currentRef, 10
dumpBinaryCode, 9
generateBinary, 9
main, 9
parseAsm, 9
resolveReferences, 9
tabLabels, 10
tabReferences, 10

asm.h, 10
AddInstruction, 11
addLabel, 11
Clean_Memory, 12
dumpBinaryCode, 12
generateBinary, 12
parseAsm, 12
resolveReferences, 12
TAILLE_BUFFER, 11

buffer
asm.c, 9

Clean_Memory
asm.c, 9
asm.h, 12

codeSegment
asm.c, 9

currentInst
asm.c, 10

currentLabel
asm.c, 10

currentRef
asm.c, 10

DEBUG
debug.h, 13
debug.h, 12
DEBUG, 13

dumpBinaryCode
asm.c, 9
asm.h, 12

generateBinary
asm.c, 9
asm.h, 12

I_ADD
Instructions.h, 14

I_AND
Instructions.h, 14

I_DIV
Instructions.h, 14

I_EQI
Instructions.h, 15

I_EQS
Instructions.h, 15

I_EQ
Instructions.h, 15

I_HALT
Instructions.h, 15

I_JF
Instructions.h, 15

I_MULT
Instructions.h, 15

I_NOT
Instructions.h, 15

I_OR
Instructions.h, 16

I_POP
Instructions.h, 16

I_PUSHI
Instructions.h, 16

I_PUSH
Instructions.h, 16

I_SUB
Instructions.h, 16

I_XOR
Instructions.h, 16

I_J
Instructions.h, 15

Instructions.h, 13
I_ADD, 14
I_AND, 14
I_DIV, 14
I_EQI, 15

- [I_EQS, 15](#)
 - [I_EQ, 15](#)
 - [I_HALT, 15](#)
 - [I_JF, 15](#)
 - [I_MULT, 15](#)
 - [I_NOT, 15](#)
 - [I_OR, 16](#)
 - [I_POP, 16](#)
 - [I_PUSH, 16](#)
 - [I_PUSH, 16](#)
 - [I_SUB, 16](#)
 - [I_XOR, 16](#)
 - [I_J, 15](#)
- label
 - [t_label, 5](#)
- main
 - [asm.c, 9](#)
- parseAsm
 - [asm.c, 9](#)
 - [asm.h, 12](#)
- resolveReferences
 - [asm.c, 9](#)
 - [asm.h, 12](#)
- [t_label, 5](#)
 - [addr, 5](#)
 - [label, 5](#)
- TAILLE_BUFFER
 - [asm.h, 11](#)
- tabLabels
 - [asm.c, 10](#)
- tabReferences
 - [asm.c, 10](#)